GPT2 Implementation

Introduction

For this project I have implemented the GPT2 transformer. I have followed Andrej Karpathy video called “Lets reproduce GPT-2 (124M)” video. I chose this for my project for the following reasons: I wanted to gain understanding how to work with PyTorch (I have written many Neural Networks throughout this video), I wanted to understand in depth the architecture of GPT2 and I enjoyed working step by step through the project.

What has been implemented

In my project I have recreated the whole GPT2 architecture. I have implemented the following things:

1. Multi Head Self Attention:

```python
class CausalSelfAttention(nn.Module):
    def __init__(self, config):
        super().__init__()
        assert config.n_embd % config.n_head == 0

        # key, query and value projection for all heads in a batch
        self.c_attn = nn.Linear(config.n_embd, 3 * config.n_embd)

        # output projection
        self.c_proj = nn.Linear(config.n_embd, config.n_embd)

        # regulization
        self.n_head = config.n_head
        self.n_embd = config.n_embd

        # more of a mask rather than a bias
        self.register_buffer("bias", torch.tril(torch.ones(config.block_size, config.block_size)).view(1,1,config.block_size, config.block_size))

    def forward(self, x):
        B, T, C = x.size()

        # create queries keys and values. Each have a shape of (1, 4, 768)
        qkv = self.c_attn(x)
        q, k, v = qkv.split(self.n_embd, dim=2)

        # multi head attention happens here. we split qkv into multiple heads
        # here 12 attention heads are created
        k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
        q = q.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
        v = v.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)

        # attention gets created
        att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(k.size(-1)))
        att = att.masked_fill(self.bias[:,:,:T,:T] == 0, float('-inf'))
        att = F.softmax(att, dim=-1)
        y = att @ v
        y = y.transpose(1, 2).contiguous().view(B, T, C)
        y = self.c_proj(y)
        return y
```

2. MLP layer:

```python
class MLP(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.c_fc = nn.Linear(config.n_embd, 4 * config.n_embd)
        self.gelu = nn.GELU(approximate='tanh')
        self.c_proj = nn.Linear(4 * config.n_embd, config.n_embd)

    def forward(self, x):
        x = self.c_fc(x)
        x = self.gelu(x)
        x = self.c_proj(x)
        return x
```

3. GPT2 transformer:

```python
class Block(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.ln_1 = nn.LayerNorm(config.n_embd)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = nn.LayerNorm(config.n_embd)
        self.mlp = MLP(config)

    def forward(self, x):
        x = x + self.attn(self.ln_1(x))
        x = x + self.mlp(self.ln_2(x))
        return x
```

4. GPT2 whole architecture:

```python
class GPT(nn.Module):
    def __init__(self, config):
        super().__init__()

        self.config = config

        self.transformer = nn.ModuleDict(dict(
            # token embedding. converting each token id into a vector of 768 numbers
            wte = nn.Embedding(config.vocab_size, config.n_embd),
            # positional embedding. converting each position into a a vector of 768 numbers
            wpe = nn.Embedding(config.block_size, config.n_embd),
            h = nn.ModuleList([Block(config) for _ in range(config.n_layer)]),
            ln_f = nn.LayerNorm(config.n_embd)
        ))

        self.lm_head = nn.Linear(config.n_embd, config.vocab_size, bias=False)

    def forward(self, idx):
        # B, T, C -> Batch = number of independent seqeunces being processed at the same time. T = number of tokens in each sequence. Channel = number of dimensions
        B, T = idx.size()
        assert T <= self.config.block_size
        pos = torch.arange(0, T, dtype=torch.long, device=idx.device)
        pos_emb = self.transformer.wpe(pos)
        tok_emb = self.transformer.wte(idx)
        # using broadcasting to add corresponding positional embeddings to each token embedding
        x = tok_emb + pos_emb

        # forward the blocks of the transformer
        for block in self.transformer.h:
            x = block(x)

        # apply layer norm to normalize the values of a tokens vector
        x = self.transformer.ln_f(x)
        logits = self.lm_head(x)
        return logits
```

What I have learned

From writing this project I fully understood the whole picture of GPT2. As a PhD candidate it's vital for my research to have a basic understanding (at least for now) of how to implement language generating models. This project has given me this understanding and gave me a good understanding of how PyTorch works and how implementing Neural Networks and Transformers work.

Future work

The limitation with this project is that I have loaded the already trained weights from hugging face:

```python
# loading the weights
@classmethod
def from_pretrained(cls, model_type):
    """Loads pretrained GPT-2 model weights from huggingface"""
    assert model_type in {'gpt2', 'gpt2-medium', 'gpt2-large', 'gpt2-xl'}
    from transformers import GPT2LMHeadModel
    print("loading weights from pretrained gpt: %s" % model_type)

    # n_layer, n_head and n_embd are determined from model_type
    config_args = {
        'gpt2':         dict(n_layer=12, n_head=12, n_embd=768),  # 124M params
        'gpt2-medium':  dict(n_layer=24, n_head=16, n_embd=1024), # 350M params
        'gpt2-large':   dict(n_layer=36, n_head=20, n_embd=1280), # 774M params
        'gpt2-xl':      dict(n_layer=48, n_head=25, n_embd=1600), # 1558M params
    }[model_type]
    config_args['vocab_size'] = 50257 # always 50257 for GPT model checkpoints
    config_args['block_size'] = 1024 # always 1024 for GPT model checkpoints
    # create a from-scratch initialized minGPT model
    config = GPTConfig(**config_args)
    model = GPT(config)
    sd = model.state_dict()
    sd_keys = sd.keys()
    sd_keys = [k for k in sd_keys if not k.endswith('.attn.bias')] # discard this mask / buffer, not a param

    # init a huggingface/transformers model
    model_hf = GPT2LMHeadModel.from_pretrained(model_type)
    sd_hf = model_hf.state_dict()

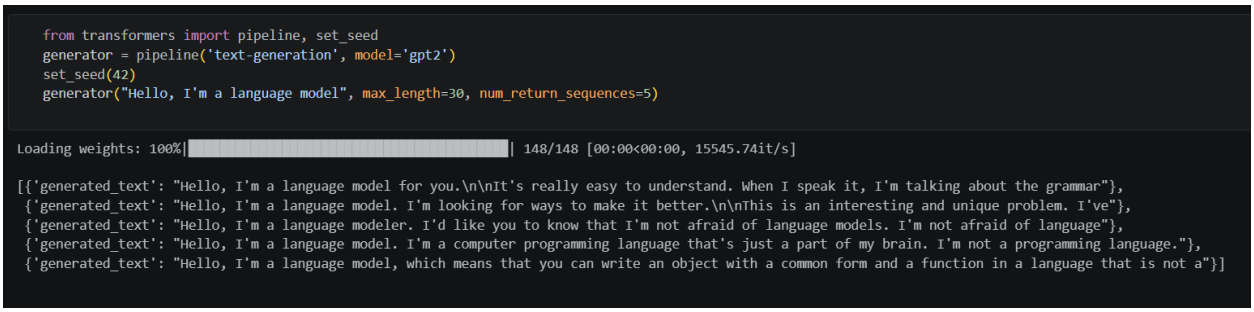
    # copy while ensuring all of the parameters are aligned and match in names and shapes
    sd_keys_hf = sd_hf.keys()
    sd_keys_hf = [k for k in sd_keys_hf if not k.endswith('.attn.masked_bias')] # ignore these, just a buffer
    sd_keys_hf = [k for k in sd_keys_hf if not k.endswith('.attn.bias')] # same, just the mask (buffer)
    transposed = ['attn.c_attn.weight', 'attn.c_proj.weight', 'mlp.c_fc.weight', 'mlp.c_proj.weight']
    # basically the openai checkpoints use a "Conv1D" module, but we only want to use a vanilla Linear
    # this means that we have to transpose these weights when we import them
    assert len(sd_keys_hf) == len(sd_keys), f"mismatched keys: {len(sd_keys_hf)} != {len(sd_keys)}"
    for k in sd_keys_hf:
        if any(k.endswith(w) for w in transposed):
            # special treatment for the Conv1D weights we need to transpose
            assert sd_hf[k].shape[::-1] == sd[k].shape
            with torch.no_grad():
                sd[k].copy_(sd_hf[k].t())
        else:
            # vanilla copy over the other parameters
            assert sd_hf[k].shape == sd[k].shape
            with torch.no_grad():
                sd[k].copy_(sd_hf[k])

    return model
```

I will continue this project and train the model by myself to understand how pretraining works and fine tuning.

Results

I compared my implementation of GPT2 to the one from hugging face. When running hugging faces GPT2 here are the results:

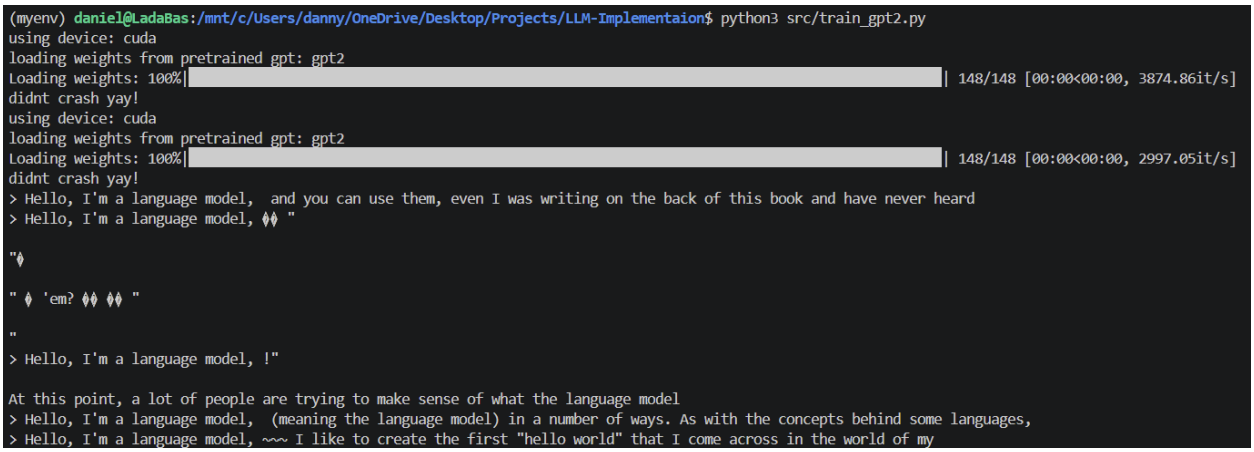


```
from transformers import pipeline, set_seed
generator = pipeline('text-generation', model='gpt2')
set_seed(42)
generator("Hello, I'm a language model", max_length=30, num_return_sequences=5)
```

```
Loading weights: 100%|| 148/148 [00:00<00:00, 15545.74it/s]

[{'generated_text': "Hello, I'm a language model for you.\n\nIt's really easy to understand. When I speak it, I'm talking about the grammar"},
 {'generated_text': "Hello, I'm a language model. I'm looking for ways to make it better.\n\nThis is an interesting and unique problem. I've"},
 {'generated_text': "Hello, I'm a language modeler. I'd like you to know that I'm not afraid of language models. I'm not afraid of language"},
 {'generated_text': "Hello, I'm a language model. I'm a computer programming language that's just a part of my brain. I'm not a programming language."},
 {'generated_text': "Hello, I'm a language model, which means that you can write an object with a common form and a function in a language that is not a"}]
```

When I ran my code, here are the results:



```
(myenv) daniel@LadaBas:/mnt/c/Users/danny/OneDrive/Desktop/Projects/LLM-Implementaion$ python3 src/train_gpt2.py
using device: cuda
loading weights from pretrained gpt: gpt2
Loading weights: 100%| 148/148 [00:00<00:00, 3874.86it/s]
didnt crash yay!
using device: cuda
loading weights from pretrained gpt: gpt2
Loading weights: 100%| 148/148 [00:00<00:00, 2997.05it/s]
didnt crash yay!
> Hello, I'm a language model,  and you can use them, even I was writing on the back of this book and have never heard
> Hello, I'm a language model, ♦♦ "

"♦

" ♦ 'em? ♦♦ ♦♦ "

"
> Hello, I'm a language model, !"

At this point, a lot of people are trying to make sense of what the language model
> Hello, I'm a language model,  (meaning the language model) in a number of ways. As with the concepts behind some languages,
> Hello, I'm a language model, ~~~ I like to create the first "hello world" that I come across in the world of my
```

At first, I was confused about why the results are different. The weights are the same so I thought the results should be the same. However, after further research I understood that it has to do with differences in text-generation settings and random sampling not because the architecture of my GPT2 is wrong.